

SENAI - CRTI

Apostila

Listagem, Busca, Paginação e CRUD de Publicações

Projeto guiado: DummyJSON — evolução do projeto de autenticação com o Feed de publicações internas (listagem, busca, paginação e CRUD completo), passo a passo para o aluno replicar sozinho

Interação com APIs — Sprint 3

Sumário

- 1. Introdução e pré-requisitos
- 2. Conceitos básicos
- 3. Passo a passo: criando o projeto pelo terminal
- 4. Código-fonte completo, arquivo por arquivo
- 5. Executando e testando a aplicação
- 6. Testes automatizados
- 7. Build de produção

1. Introdução e pré-requisitos

Nesta apostila você vai construir, do zero, a aplicação Angular DummyJSON completa: a tela de login com autenticação JWT, a área protegida de perfil e o Feed de publicações internas, com listagem paginada, busca dinâmica e CRUD completo (criar, editar e excluir publicações). Todas as operações conversam com a API pública DummyJSON, tanto para autenticação (/auth/login, /auth/me) quanto para publicações (/posts).

Siga os capítulos na ordem. Cada comando de terminal e cada arquivo de código aparecem completos, prontos para copiar.

Pré-requisitos

- Node.js instalado (versão LTS mais recente).
- Angular CLI instalado globalmente.
- Um editor de código (ex.: VS Code).

Para instalar o Angular CLI globalmente, caso ainda não tenha:

```
$ npm install -g @angular/cli
```

Para conferir se tudo está pronto, rode os três comandos abaixo. Eles devem responder com um número de versão, sem erro:

```
$ node --version
$ npm --version
$ ng version
```

2. Conceitos básicos

Antes de programar, é importante entender o papel de cada peça que vamos usar.

HttpClient

É o serviço oficial do Angular para fazer requisições HTTP. Nesta apostila usamos `post()` e `patch()` para criar e editar publicações, `get()` para listar e buscar, e `delete()` para excluir.

Autenticação × Autorização

Autenticação é o processo de confirmar quem o usuário é (login com usuário e senha, devolvendo um token). Autorização é decidir o que esse usuário autenticado pode acessar. O `AuthService` cuida da autenticação; o `Route Guard` cuida da autorização de rotas.

Token JWT e cabeçalho Bearer

Um JWT (JSON Web Token) é uma string assinada que a API devolve depois de um login válido. A partir daí, toda requisição a uma rota protegida precisa enviar esse token no cabeçalho HTTP Authorization, no formato Authorization: Bearer <token>.

AuthService

Uma classe decorada com `@Service()` — o decorator padrão do Angular 22 para services singleton — responsável por fazer o login, guardar a sessão (token + dados do usuário) e expor essa sessão para o resto da aplicação através de signals.

HTTP Interceptor

Uma função (`HttpInterceptorFn`) que intercepta toda requisição HTTP feita pela aplicação antes dela sair. Usamos o interceptor para adicionar o cabeçalho Authorization automaticamente e para reagir a uma resposta 401 Unauthorized deslogando o usuário.

Route Guard

Uma função (`CanActivateFn`) que o Angular Router consulta antes de ativar uma rota protegida. Usamos o `authGuard` para impedir que as rotas `/perfil` e `/feed` sejam abertas por alguém que não fez login.

PostsService e o estado do Feed

Assim como o `AuthService` concentra a autenticação, o `PostsService` concentra todo o estado da tela de publicações: a lista atual (posts), a paginação (total e skip), o termo de busca (termoBusca) e os estados de carregando e erro — tudo como signals. Os componentes apenas leem esses signals e chamam métodos do service; nenhuma lógica de HTTP fica no componente.

Paginação com limit e skip

A API de posts da DummyJSON pagina resultados através dos parâmetros de query `limit` (quantos itens por página) e `skip` (quantos itens pular). A resposta inclui `total`, `skip` e `limit`, usados para calcular quantas páginas existem e habilitar ou desabilitar os botões de navegação.

Busca dinâmica com debounceTime

Em vez de um botão de "buscar", o campo de busca do Feed reage a cada tecla digitada através de valueChanges do FormControl. O operador debounceTime do RxJS espera 400ms de silêncio antes de disparar a busca, e distinctUntilChanged evita repetir a mesma busca — assim a API não é chamada a cada tecla, só quando o usuário para de digitar.

Edição inline com um signal de controle

Em vez de abrir uma tela ou um modal separado para editar, cada publicação da lista pode se transformar no próprio formulário de edição. Um signal (editandoId) guarda o id da publicação em edição no momento; o template usa esse valor para decidir, item por item, se mostra o texto normal ou o formulário de edição.

CRUD com PATCH

Diferente do PUT (que substitui o recurso inteiro), o método PATCH atualiza apenas os campos enviados — no caso, title e body. É o método usado por editar() no PostsService, em POST /posts/:id.

Confirmação antes de excluir

Excluir uma publicação é uma ação destrutiva e difícil de desfazer, por isso o componente chama window.confirm() antes de enviar o DELETE — se o usuário cancelar a caixa de diálogo do navegador, nada é enviado à API.

Feedback visual (toast)

Depois de criar, editar ou excluir uma publicação, uma pequena mensagem colorida (toast) aparece no rodapé da tela por alguns segundos, confirmando sucesso ou avisando sobre um erro. É controlada por um signal local do componente, sem depender de nenhuma biblioteca externa.

Um detalhe importante da DummyJSON: ela não persiste escritas

A DummyJSON é uma API de simulação: POST /posts/add, PATCH /posts/:id e DELETE /posts/:id respondem como se a operação tivesse funcionado, mas nada é realmente salvo no banco de dados da API. Na prática, isso tem uma consequência direta: o id devolvido por um POST /posts/add (por exemplo, 252) não existe de verdade na base, então um PATCH ou DELETE posterior para esse mesmo id retornaria 404 Not Found.

Por isso, o PostsService guarda, em memória, os ids das publicações criadas durante a sessão atual (foiCriadoNestaSessao). Ao editar ou excluir uma publicação com esse id, a operação é aplicada só na lista local, sem chamar a API; publicações que já existiam de verdade na DummyJSON continuam sendo editadas e excluídas normalmente através de PATCH e DELETE reais.

Zoneless (Angular 21+)

Desde a versão 21, o Angular não depende mais da biblioteca Zone.js para saber quando atualizar a tela. Em vez disso, a atualização acontece diretamente quando um Signal muda

de valor — já vem assim por padrão em projetos novos, sem exigir nenhuma configuração extra.

Resumo: ação da tela → mecanismo → onde acontece

- Preencher o formulário e clicar em "Entrar" → POST → authService.login(credenciais)
- Digitar no campo de busca do Feed → debounce de 400ms → postsService.buscar(termo)
- Clicar em "Publicar" → POST /posts/add → postsService.criar(post)
- Clicar em "Editar" e depois "Salvar" → PATCH /posts/:id (ou só local, se for post da sessão) → postsService.editar / atualizarNaLista
- Clicar em "Excluir" e confirmar → DELETE /posts/:id (ou só local, se for post da sessão) → postsService.remove / removeDaLista
- Clicar em "Anterior" / "Próxima" → recalcula skip → postsService.paginaAnterior / proximaPagina
- A API responde 401 em qualquer chamada → desloga e redireciona ao /login → authInterceptor

3. Passo a passo: criando o projeto pelo terminal

Abra o terminal na pasta onde deseja criar o projeto e siga os comandos na ordem.

3.1 Criar o projeto Angular

```
$ ng new DummyJSON --style=css --routing=true --ssr=false --defaults
```

O que cada opção faz: --style=css usa CSS puro para os estilos; --routing=true porque a aplicação tem mais de uma tela (login, perfil e feed) e precisa de rotas; --ssr=false porque a aplicação roda só no navegador (client-side); --defaults aceita as demais respostas padrão automaticamente (o Angular 22 já cria o projeto zoneless por padrão).

3.2 Entrar na pasta do projeto

```
$ cd DummyJSON
```

3.3 Gerar o model (interface) de Usuário

```
$ ng generate interface core/models/usuario
```

Esse comando cria o arquivo src/app/core/models/usuario.ts, com uma interface vazia. Vamos preencher com três interfaces — Login, UsuarioAutenticado e PerfilCompleto — no capítulo 4.

3.4 Gerar o AuthService

```
$ ng generate service core/services/auth
```

Esse comando cria dois arquivos: `src/app/core/services/auth.ts` (o service, já decorado com `@Service()`) e `src/app/core/services/auth.spec.ts`. Renomeie a classe gerada para `AuthService` (o arquivo continua `auth.ts`, seguindo a convenção do Angular 22 de não usar sufixos de tipo nos nomes de arquivo).

3.5 Gerar o Interceptor de autenticação

```
$ ng generate interceptor core/interceptors/auth --skip-tests
```

Cria `src/app/core/interceptors/auth-interceptor.ts`, já como uma função `HttpInterceptorFn` chamada `authInterceptor`.

3.6 Gerar o Route Guard de autenticação

```
$ ng generate guard core/auth --skip-tests
```

O CLI pergunta qual tipo de guard você quer criar — escolha `CanActivate`. Isso cria `src/app/core/auth-guard.ts`, já como uma função `CanActivateFn` chamada `authGuard`.

3.7 Gerar as telas de Login e Perfil

```
$ ng generate component features/login --skip-tests
$ ng generate component features/perfil --skip-tests
```

Cada comando cria uma pasta com três arquivos: o componente (`.ts`), o template (`.html`) e o estilo (`.css`) — o Angular CLI 22 não gera mais o sufixo "component" em nenhum desses nomes.

3.8 Gerar o model, o service e a tela de publicações (Feed)

```
$ ng generate interface core/models/post
$ ng generate service core/services/posts
$ ng generate component features/feed --skip-tests
```

O mesmo padrão usado para autenticação se repete aqui: uma interface para o formato dos dados, um service (já com `@Service()`) concentrando toda a comunicação com a API e o estado da tela, e um componente para a interface visual. Renomeie a classe gerada do service para `PostsService` (arquivo continua `posts.ts`).

3.9 Conectar rotas, HttpClient, Interceptor e navegação

Os comandos acima criaram os arquivos, mas ainda não os conectaram entre si. No capítulo 4, vamos editar manualmente:

- `src/app/app.config.ts` — registrar o `HttpClient` com o interceptor de autenticação.
- `src/app/app.routes.ts` — registrar as rotas `/login`, `/perfil` e `/feed`, as duas últimas protegidas pelo guard.
- `src/app/features/perfil/perfil.html` — adicionar um link de navegação para `/feed`.

- `src/app/features/feed/feed.html` — adicionar um link de navegação de volta para `/perfil`.

Não existe comando de terminal para essa etapa: conectar as peças é sempre feito no código.

3.10 Estrutura final de pastas

Depois de todos os comandos, e com os arquivos preenchidos (próximo capítulo), a estrutura deve ficar assim:

```
src/  
  main.ts  
  app/  
    app.config.ts  
    app.routes.ts  
    app.ts  
    app.html  
    app.css  
    app.spec.ts  
  core/  
    auth-guard.ts  
    interceptors/  
      auth-interceptor.ts  
    models/  
      usuario.ts  
      post.ts  
    services/  
      auth.ts  
      auth.spec.ts  
      posts.ts  
      posts.spec.ts  
  features/  
    login/  
      login.ts  
      login.html  
      login.css  
    perfil/  
      perfil.ts  
      perfil.html  
      perfil.css  
    feed/  
      feed.ts  
      feed.html  
      feed.css
```

4. Código-fonte completo, arquivo por arquivo

A seguir está o conteúdo final de cada arquivo. Abra cada um no editor e substitua o conteúdo gerado pelo Angular CLI pelo código abaixo.

4.1 src/main.ts

Ponto de entrada da aplicação. Inicializa (bootstrap) o componente principal App usando as configurações definidas em appConfig. Este arquivo é gerado pronto pelo Angular CLI — não precisa alterá-lo.

Arquivo: src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

4.2 src/app/app.config.ts

Configura os providers globais da aplicação: provideRouter(routes) habilita a navegação entre telas; provideHttpClient(withInterceptors([authInterceptor])) habilita o HttpClient já registrando o interceptor de autenticação — a partir daqui, toda requisição feita pela aplicação passa primeiro pelo authInterceptor.

Arquivo: src/app/app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';

import { routes } from './app.routes';
import { authInterceptor } from './core/interceptors/auth-interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
  ],
};
```

4.3 src/app/app.routes.ts

Define três rotas: /login (pública), /perfil e /feed (ambas protegidas pelo authGuard no campo canActivate). A rota raiz redireciona para /login, e qualquer URL desconhecida (**)

também. Todos os componentes são carregados de forma lazy (loadComponent), ou seja, o código de cada tela só é baixado quando o usuário navega até ela.

Arquivo: src/app/app.routes.ts

```
import { Routes } from '@angular/router';

import { authGuard } from '../core/auth-guard';

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'login' },
  {
    path: 'login',
    loadComponent: () => import('../features/login/login').then((m) => m.Login),
  },
  {
    path: 'perfil',
    canActivate: [authGuard],
    loadComponent: () => import('../features/perfil/perfil').then((m) => m.Perfil),
  },
  {
    path: 'feed',
    canActivate: [authGuard],
    loadComponent: () => import('../features/feed/feed').then((m) => m.Feed),
  },
  { path: '**', redirectTo: '' },
];
```

4.4 src/app/core/models/usuario.ts

Login descreve o payload enviado ao fazer login. UsuarioAutenticado descreve o formato real que a DummyJSON devolve em POST /auth/login — incluindo accessToken e refreshToken. PerfilCompleto descreve o formato devolvido pela rota protegida GET /auth/me.

Arquivo: src/app/core/models/usuario.ts

```
export interface Login {
  username: string;
  password: string;
}

export interface UsuarioAutenticado {
  id: number;
  username: string;
  email: string;
  firstName: string;
  lastName: string;
  gender: string;
}
```

```

    image: string;
    accessToken: string;
    refreshToken: string;
  }

export interface PerfilCompleto {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  role: string;
  image: string;
}

```

4.5 src/app/core/services/auth.ts

O AuthService concentra toda a lógica de autenticação. O signal privado usuario começa com o valor salvo em sessionStorage (se existir). login() faz o POST e devolve o Observable — quem chama decide quando assinar. salvarSessao() persiste a sessão. buscarPerfilCompleto() faz uma chamada GET separada e autenticada. obterToken() é usada pelo interceptor.

Arquivo: src/app/core/services/auth.ts

```

import { Injectable, computed, inject, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';

import { Login, PerfilCompleto, UsuarioAutenticado } from '../models/usuario';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);

  private readonly API_LOGIN = 'https://dummyjson.com/auth/login';
  private readonly API_PERFIL = 'https://dummyjson.com/auth/me';
  private readonly CHAVE_SESSAO = 'dummyjson-usuario';

  private usuario = signal<UsuarioAutenticado | null>(this.carregarSessaoSalva());

  usuarioAtual = computed(() => this.usuario());
  estaAutenticado = computed(() => this.usuario() !== null);

  login(credenciais: Login) {
    return this.http.post<UsuarioAutenticado>(this.API_LOGIN, credenciais);
  }

  buscarPerfilCompleto() {
    return this.http.get<PerfilCompleto>(this.API_PERFIL);
  }
}

```

```

}

salvarSessao(usuario: UsuarioAutenticado) {
  this.usuario.set(usuario);
  sessionStorage.setItem(this.CHAVE_SESSAO, JSON.stringify(usuario));
}

logout() {
  this.usuario.set(null);
  sessionStorage.removeItem(this.CHAVE_SESSAO);
}

obterToken(): string | null {
  return this.usuario()?.accessToken ?? null;
}

private carregarSessaoSalva(): UsuarioAutenticado | null {
  const dadosSalvos = sessionStorage.getItem(this.CHAVE_SESSAO);

  if (!dadosSalvos) {
    return null;
  }

  try {
    return JSON.parse(dadosSalvos) as UsuarioAutenticado;
  } catch {
    return null;
  }
}
}

```

4.6 src/app/core/interceptors/auth-interceptor.ts

Intercepta toda requisição feita pela aplicação. Se houver um token salvo, clona a requisição original adicionando o cabeçalho Authorization — a requisição em si nunca é modificada diretamente, porque objetos `HttpRequest` são imutáveis no Angular. Depois de repassar a requisição, o `catchError` observa a resposta: se o status for 401, desloga o usuário e redireciona para `/login` antes de propagar o erro adiante.

Arquivo: src/app/core/interceptors/auth-interceptor.ts

```

import { inject } from '@angular/core';
import { HttpInterceptorFn } from '@angular/common/http';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';

import { AuthService } from '../../services/auth';

```

```

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  const token = authService.obterToken();

  const requisicaoAutenticada = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(requisicaoAutenticada).pipe(
    catchError((erro) => {
      if (erro.status === 401) {
        authService.logout();
        router.navigateByUrl('/login');
      }

      return throwError(() => erro);
    }),
  );
};

```

4.7 src/app/core/auth-guard.ts

Consultado pelo Angular Router antes de ativar as rotas /perfil e /feed. Se houver sessão ativa, libera a navegação devolvendo true. Caso contrário, devolve uma UrlTree apontando para /login.

Arquivo: src/app/core/auth-guard.ts

```

import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

import { AuthService } from '../services/auth';

export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.estaAutenticado()) {
    return true;
  }

  return router.createUrlTree(['/login']);
};

```

4.8 src/app/features/login/login.ts

Formulário reativo com dois campos obrigatórios (username e password). Se for válido, chama `authService.login()` — no sucesso, salva a sessão e navega para `/perfil`; no erro, distingue um 400 (credenciais erradas) de qualquer outro problema.

Arquivo: `src/app/features/login/login.ts`

```
import { Component, inject, signal } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { HttpResponse } from '@angular/common/http';
import { Router } from '@angular/router';

import { AuthService } from '../../core/services/auth';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css'],
})
export class Login {
  private authService = inject(AuthService);
  private router = inject(Router);

  carregando = signal(false);
  erro = signal('');

  formulario = new FormGroup({
    username: new FormControl('', { nullable: true, validators: [Validators.required] }),
    password: new FormControl('', { nullable: true, validators: [Validators.required] }),
  });

  entrar() {
    if (this.formulario.invalid) {
      this.formulario.markAllAsTouched();
      this.erro.set('Preencha usuário e senha.');
      return;
    }

    this.carregando.set(true);
    this.erro.set('');

    this.authService.login(this.formulario.getRawValue()).subscribe({
      next: (usuario) => {
        this.authService.salvarSessao(usuario);
        this.carregando.set(false);
        this.router.navigateByUrl('/perfil');
      },
      error: (erro: HttpResponse) => {
```

```

        this.carregando.set(false);
        this.erro.set(
            erro.status === 400
                ? 'Usuário ou senha inválidos.'
                : 'Não foi possível fazer login. Tente novamente.',
        );
    },
    });
}
}
}

```

4.9 src/app/features/login/login.html

Template do login: um cartão centralizado com uma dica visível dos dados de teste, os dois campos do formulário, a mensagem de erro (quando existir) e o botão de envio, desabilitado durante o carregando().

Arquivo: src/app/features/login/login.html

```

<div class="container">
  <form class="cartao" [formGroup]="formulario" (ngSubmit)="entrar()">
    <h1>DummyJSON Login</h1>
    <p class="dica">
      Usuário de teste: <strong>emilys</strong> · Senha: <strong>emilyspass</strong>
    </p>

    <label>
      Usuário
      <input type="text" formControlName="username" placeholder="username" />
    </label>

    <label>
      Senha
      <input type="password" formControlName="password" placeholder="password" />
    </label>

    @if (erro()) {
      <p class="erro">{{ erro() }}</p>
    }

    <button type="submit" [disabled]="carregando()">
      {{ carregando() ? 'Entrando...' : 'Entrar' }}
    </button>
  </form>
</div>

```

4.10 src/app/features/login/login.css

Estilos da tela de login: cartão branco centralizado na tela, caixa de dica em azul claro, campos empilhados e botão que muda de cor ao passar o mouse ou fica acinzentado quando desabilitado.

Arquivo: src/app/features/login/login.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

:host {
  display: block;
  background: #f5f7fb;
  min-height: 100vh;
}

.container {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  padding: 20px;
}

.cartao {
  display: flex;
  flex-direction: column;
  gap: 14px;
  width: 100%;
  max-width: 360px;
  background: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
}

h1 {
  text-align: center;
  font-size: 22px;
  color: #222;
}

.dica {
  text-align: center;
  font-size: 13px;
}
```

```
    color: #666;
    background: #f0f4ff;
    padding: 10px;
    border-radius: 8px;
  }

  label {
    display: flex;
    flex-direction: column;
    gap: 6px;
    font-size: 14px;
    color: #444;
    font-weight: bold;
  }

  input {
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 8px;
    font-size: 15px;
    font-weight: normal;
  }

  button {
    border: none;
    background: #2563eb;
    color: white;
    padding: 14px;
    border-radius: 8px;
    cursor: pointer;
    font-weight: bold;
    font-size: 15px;
  }

  button:hover:not(:disabled) {
    background: #1d4ed8;
  }

  button:disabled {
    background: #93c5fd;
    cursor: not-allowed;
  }

  .erro {
    color: #dc2626;
    font-weight: bold;
    font-size: 14px;
  }
```


4.11 src/app/features/perfil/perfil.ts

A tela protegida. usuario expõe a sessão salva no login (leitura direta do signal do AuthService). No construtor, chama carregarPerfilCompleto(), que faz uma nova chamada GET a /auth/me — propositalmente separada dos dados já salvos no login, para provar que o interceptor está injetando o token sozinho nessa nova requisição. sair() limpa a sessão e volta para /login.

Arquivo: src/app/features/perfil/perfil.ts

```
import { Component, inject, signal } from '@angular/core';
import { Router, RouterLink } from '@angular/router';

import { AuthService } from '../../../core/services/auth';
import { PerfilCompleto } from '../../../core/models/usuario';

@Component({
  selector: 'app-perfil',
  imports: [RouterLink],
  templateUrl: './perfil.html',
  styleUrls: ['./perfil.css'],
})
export class Perfil {
  private authService = inject(AuthService);
  private router = inject(Router);

  usuario = this.authService.usuarioAtual;

  perfilCompleto = signal<PerfilCompleto | null>(null);
  carregando = signal(true);
  erro = signal('');

  constructor() {
    this.carregarPerfilCompleto();
  }

  carregarPerfilCompleto() {
    this.carregando.set(true);
    this.erro.set('');

    this.authService.buscarPerfilCompleto().subscribe({
      next: (perfil) => {
        this.perfilCompleto.set(perfil);
        this.carregando.set(false);
      },
      error: () => {
        this.erro.set('Não foi possível confirmar sua sessão com a API. Tente novamente.');
```

```

    });
  }

  sair() {
    this.authService.logout();
    this.router.navigateByUrl('/login');
  }
}

```

4.12 src/app/features/perfil/perfil.html

Template da área protegida: um cabeçalho com o link para o Feed e o botão Sair, os dados da sessão local salvos no login, e uma seção separada — com seu próprio estado de carregando/erro/sucesso — que mostra o resultado da chamada autenticada a /auth/me.

Arquivo: src/app/features/perfil/perfil.html

```

<div class="container">
  <div class="cartao">
    <header class="topo">
      <h1>Área protegida</h1>
      <nav class="acoes-topo">
        <a routerLink="/feed">Ver publicações</a>
        <button type="button" class="btn-sair" (click)="sair()">Sair</button>
      </nav>
    </header>

    @if (usuario(); as sessao) {
      <section class="sessao">
        <img [src]="sessao.image" [alt]="sessao.username" />
        <div>
          <p><strong>Sessão local (salva no login):</strong></p>
          <p>{{ sessao.firstName }} {{ sessao.lastName }} ({{ sessao.username }})</p>
          <p>{{ sessao.email }}</p>
        </div>
      </section>
    }

    <hr />

    <h2>Chamada autenticada a /auth/me</h2>
    <p class="explicacao">
      Esta seção não usa os dados salvos no login – ela faz uma nova requisição HTTP,
      autenticada automaticamente pelo interceptor com o token guardado na sessão.
    </p>

    @if (carregando()) {
      <p>Confirmando sessão com a API...</p>
    }
  </div>

```

```

    @if (erro()) {
      <section class="mensagem-erro">
        <p>{{ erro() }}</p>
        <button type="button" (click)="carregarPerfilCompleto()">Tentar novamente</button>
      </section>
    }

    @if (perfilCompleto(); as perfil) {
      <ul class="detalhes">
        <li><strong>Perfil (role):</strong> {{ perfil.role }}</li>
        <li><strong>Nome completo:</strong> {{ perfil.firstName }} {{ perfil.lastName }}</li>
        <li><strong>E-mail:</strong> {{ perfil.email }}</li>
      </ul>
    }
  </div>
</div>

```

4.13 src/app/features/perfil/perfil.css

Estilos da área protegida: cartão branco, cabeçalho com a navegação (link para o Feed e botão Sair) alinhada à direita, avatar circular, lista de detalhes do perfil e a caixa de erro com o botão de tentar novamente.

Arquivo: src/app/features/perfil/perfil.css

```

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

:host {
  display: block;
  background: #f5f7fb;
  min-height: 100vh;
}

.container {
  display: flex;
  justify-content: center;
  padding: 40px 20px;
}

.cartao {
  width: 100%;
  max-width: 520px;
  background: white;
}

```

```
padding: 30px;
border-radius: 12px;
box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
}
```

```
.topo {
display: flex;
justify-content: space-between;
align-items: center;
margin-bottom: 20px;
}
```

```
h1 {
font-size: 20px;
color: #222;
}
```

```
h2 {
font-size: 16px;
color: #222;
margin-bottom: 8px;
}
```

```
.acoes-topo {
display: flex;
align-items: center;
gap: 14px;
}
```

```
.acoes-topo a {
color: #2563eb;
font-size: 14px;
font-weight: bold;
text-decoration: none;
}
```

```
.btn-sair {
border: none;
background: #fee2e2;
color: #b91c1c;
padding: 8px 16px;
border-radius: 8px;
cursor: pointer;
font-weight: bold;
font-size: 13px;
}
```

```
.btn-sair:hover {
background: #fecaca;
}
```

```
}

.sessao {
  display: flex;
  gap: 14px;
  align-items: center;
}

.sessao img {
  width: 56px;
  height: 56px;
  border-radius: 50%;
}

.sessao p {
  color: #444;
  font-size: 14px;
  margin: 2px 0;
}

hr {
  border: none;
  border-top: 1px solid #eee;
  margin: 20px 0;
}

.explicacao {
  color: #666;
  font-size: 13px;
  margin-bottom: 14px;
}

.detalhes {
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.detalhes li {
  color: #444;
  font-size: 14px;
}

.mensagem-erro {
  display: flex;
  flex-direction: column;
  gap: 10px;
  align-items: flex-start;
}
```

```

}

.mensagem-erro p {
  color: #dc2626;
  font-weight: bold;
  font-size: 14px;
}

.mensagem-erro button {
  border: none;
  background: #2563eb;
  color: white;
  padding: 8px 16px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
  font-size: 13px;
}

```

4.14 src/app/core/models/post.ts

Post descreve o formato de uma publicação devolvida pela DummyJSON. PostsResposta descreve a resposta paginada de GET /posts e GET /posts/search (posts, total, skip, limit). NovoPost descreve o payload enviado ao criar ou editar uma publicação.

Arquivo: src/app/core/models/post.ts

```

export interface Post {
  id: number;
  title: string;
  body: string;
  userId: number;
  tags?: string[];
}

export interface PostsResposta {
  posts: Post[];
  total: number;
  skip: number;
  limit: number;
}

export interface NovoPost {
  title: string;
  body: string;
  userId: number;
}

```

4.15 src/app/core/services/posts.ts

O PostsService concentra todo o estado do Feed: a lista (posts), a paginação (total, skip e os signals computados paginaAtual/totalPaginas/temPaginaAnterior/temProximaPagina) e o termo de busca. carregar() decide entre GET /posts e GET /posts/search conforme haja um termo digitado. criar(), editar() e remover() chamam a API; adicionarNaLista(), atualizarNaLista() e removerDaLista() mantêm a lista local sincronizada, já que a DummyJSON não persiste as escritas de verdade. foiCriadoNestaSessao() evita chamar PATCH/DELETE para um id que só existe localmente.

Arquivo: src/app/core/services/posts.ts

```
import { Injectable, computed, inject, signal } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';

import { NovoPost, Post, PostsResposta } from '../models/post';

const ITENS_POR_PAGINA = 10;

@Injectable({
  providedIn: 'root',
})
export class PostsService {
  private http = inject(HttpClient);

  private readonly API_POSTS = 'https://dummyjson.com/posts';

  posts = signal<Post[]>([]);
  // Publicações criadas nesta sessão: a DummyJSON não as conhece, então elas
  // ficam fora do que /posts e /posts/search devolvem. Mantemos essa lista à
  // parte para que a busca também as encontre, em vez de só os posts da API.
  postsCriados = signal<Post[]>([]);
  total = signal(0);
  skip = signal(0);
  termoBusca = signal('');
  carregando = signal(false);
  erro = signal('');

  private idsCriadosNestaSessao = new Set<number>();
  // A DummyJSON sempre responde ao POST /posts/add com o mesmo id, então geramos
  // um id local exclusivo para cada post criado, evitando que duas publicações
  // criadas na mesma sessão colidam com o mesmo id (o que quebra o "track" da lista
  // e faz editar/excluir uma delas afetar as duas).
  private proximoIdLocal = Date.now();

  paginaAtual = computed(() => Math.floor(this.skip() / ITENS_POR_PAGINA) + 1);
  totalPaginas = computed(() => Math.max(1, Math.ceil(this.total() / ITENS_POR_PAGINA)));
  temPaginaAnterior = computed(() => this.skip() > 0);
  temProximaPagina = computed(() => this.skip() + ITENS_POR_PAGINA < this.total());
```

```

// Combina os posts criados localmente (filtrados pelo termo de busca, quando
// houver um) com a página atual vinda da API, para exibição.
postsExibidos = computed(() => {
  const termo = this.termoBusca().trim().toLowerCase();
  const criados = termo
    ? this.postsCriados().filter(
      (post) =>
        post.title.toLowerCase().includes(termo) || post.body.toLowerCase().includes(termo),
    )
    : this.postsCriados();
  return [...criados, ...this.posts()];
});

carregar() {
  this.carregando.set(true);
  this.erro.set('');

  const termo = this.termoBusca().trim();
  const url = termo ? `${this.API_POSTS}/search` : this.API_POSTS;

  let params = new HttpParams().set('limit', ITENS_POR_PAGINA).set('skip', this.skip());
  if (termo) {
    params = params.set('q', termo);
  }

  this.http.get<PostsResposta>(url, { params }).subscribe({
    next: (resposta) => {
      this.posts.set(resposta.posts);
      this.total.set(resposta.total);
      this.carregando.set(false);
    },
    error: () => {
      this.erro.set('Não foi possível carregar as publicações.');
```



```

    this.skip.update((valor) => valor + ITENS_POR_PAGINA);
    this.carregar();
  }

  paginaAnterior() {
    if (!this.temPaginaAnterior()) {
      return;
    }

    this.skip.update((valor) => Math.max(0, valor - ITENS_POR_PAGINA));
    this.carregar();
  }

  criar(post: NovoPost) {
    return this.http.post<Post>(`${this.API_POSTS}/add`, post);
  }

  editar(id: number, dados: Partial<NovoPost>) {
    return this.http.patch<Post>(`${this.API_POSTS}/${id}`, dados);
  }

  remover(id: number) {
    return this.http.delete(`${this.API_POSTS}/${id}`);
  }

  // A DummyJSON simula as escritas (não persiste de verdade), então a lista local
  // precisa ser atualizada manualmente após cada operação bem-sucedida.
  adicionarNaLista(post: Post) {
    const postComIdUnico: Post = { ...post, id: this.proximoIdLocal++ };
    this.postsCriados.update((lista) => [postComIdUnico, ...lista]);
    this.total.update((valor) => valor + 1);
    this.idsCriadosNestaSessao.add(postComIdUnico.id);
  }

  atualizarNaLista(id: number, dados: Partial<Post>) {
    if (this.idsCriadosNestaSessao.has(id)) {
      this.postsCriados.update((lista) =>
        lista.map((post) => (post.id === id ? { ...post, ...dados } : post)),
      );
      return;
    }
    this.posts.update((lista) =>
      lista.map((post) => (post.id === id ? { ...post, ...dados } : post)),
    );
  }

  removerDaLista(id: number) {
    if (this.idsCriadosNestaSessao.has(id)) {

```

```

        this.postsCriados.update((lista) => lista.filter((post) => post.id !== id));
        this.idsCriadosNestaSessao.delete(id);
        this.total.update((valor) => Math.max(0, valor - 1));
        return;
    }
    this.posts.update((lista) => lista.filter((post) => post.id !== id));
    this.total.update((valor) => Math.max(0, valor - 1));
}

// O id devolvido por POST /posts/add não existe de verdade na base da DummyJSON:
// um PATCH ou DELETE para esse id retorna 404. Por isso, editar/excluir um post
// criado nesta sessão é resolvido só na lista local, sem chamar a API.
foiCriadoNestaSessao(id: number): boolean {
    return this.idsCriadosNestaSessao.has(id);
}
}

```

4.16 src/app/features/feed/feed.ts

O componente principal do Feed. No construtor, dispara o carregamento inicial e assina `valueChanges` do campo de busca com `debounceTime(400) + distinctUntilChanged()`, chamando `postsService.buscar()` a cada novo termo. `criar()`, `salvarEdicao()` e `excluir()` chamam o `PostsService` e, no sucesso (ou quando o post é local, via `foiCriadoNestaSessao`), atualizam a lista e disparam um toast.

Arquivo: src/app/features/feed/feed.ts

```

import { Component, inject, signal } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { debounceTime, distinctUntilChanged } from 'rxjs';

import { AuthService } from '../../core/services/auth';
import { PostsService } from '../../core/services/posts';
import { Post } from '../../core/models/post';

interface Toast {
    texto: string;
    tipo: 'sucesso' | 'erro';
}

@Component({
    selector: 'app-feed',
    imports: [ReactiveFormsModule, RouterLink],
    templateUrl: './feed.html',
    styleUrls: ['./feed.css'],
})
export class Feed {
    protected authService = inject(AuthService);

```

```

protected postsService = inject(PostsService);
private router = inject(Router);

buscaControl = new FormControl('', { nullable: true });

toast = signal<Toast | null>(null);
editandoId = signal<number | null>(null);

formularioNovo = new FormGroup({
  title: new FormControl('', { nullable: true, validators: [Validators.required] }),
  body: new FormControl('', { nullable: true, validators: [Validators.required] }),
});

formularioEdicao = new FormGroup({
  title: new FormControl('', { nullable: true, validators: [Validators.required] }),
  body: new FormControl('', { nullable: true, validators: [Validators.required] }),
});

constructor() {
  this.postsService.carregar();

  this.buscaControl.valueChanges
    .pipe(debounceTime(400), distinctUntilChanged())
    .subscribe((termo) => this.postsService.buscar(termo));
}

criar() {
  if (this.formularioNovo.invalid) {
    this.formularioNovo.markAllAsTouched();
    return;
  }

  const usuarioId = this.authService.usuarioAtual()?.id ?? 0;
  const { title, body } = this.formularioNovo.getRawValue();

  this.postsService.criar({ title, body, userId: usuarioId }).subscribe({
    next: (post) => {
      this.postsService.adicionarNaLista(post);
      this.formularioNovo.reset({ title: '', body: '' });
      this.mostrarToast('Publicação criada com sucesso.', 'sucesso');
    },
    error: () => this.mostrarToast('Não foi possível criar a publicação.', 'erro'),
  });
}

iniciarEdicao(post: Post) {
  this.editandoId.set(post.id);
  this.formularioEdicao.setValue({ title: post.title, body: post.body });
}

```

```
cancelarEdicao() {
  this.editandoId.set(null);
}

salvarEdicao(id: number) {
  if (this.formularioEdicao.invalid) {
    this.formularioEdicao.markAllAsTouched();
    return;
  }

  const dados = this.formularioEdicao.getRawValue();
  const finalizarEdicao = () => {
    this.postsService.atualizarNaLista(id, dados);
    this.editandoId.set(null);
    this.mostrarToast('Publicação atualizada com sucesso.', 'sucesso');
  };

  if (this.postsService.foiCriadoNestaSessao(id)) {
    finalizarEdicao();
    return;
  }

  this.postsService.editar(id, dados).subscribe({
    next: finalizarEdicao,
    error: () => this.mostrarToast('Não foi possível atualizar a publicação.', 'erro'),
  });
}

excluir(id: number) {
  if (!confirm('Excluir esta publicação?')) {
    return;
  }

  const finalizarExclusao = () => {
    this.postsService.removerDaLista(id);
    this.mostrarToast('Publicação excluída com sucesso.', 'sucesso');
  };

  if (this.postsService.foiCriadoNestaSessao(id)) {
    finalizarExclusao();
    return;
  }

  this.postsService.remover(id).subscribe({
    next: finalizarExclusao,
    error: () => this.mostrarToast('Não foi possível excluir a publicação.', 'erro'),
  });
}
```

```

sair() {
  this.authService.logout();
  this.router.navigateByUrl('/login');
}

private mostrarToast(texto: string, tipo: Toast['tipo']) {
  this.toast.set({ texto, tipo });
  setTimeout(() => this.toast.set(null), 3000);
}
}

```

4.17 src/app/features/feed/feed.html

Template do Feed: cabeçalho com link de volta ao perfil, formulário de nova publicação, campo de busca, estados de carregando/erro/lista vazia, a lista de publicações (cada item alterna entre visualização e formulário de edição inline conforme editandoId), os controles de paginação e o toast de feedback.

Arquivo: src/app/features/feed/feed.html

```

<div class="container">
  <div class="pagina">
    <header class="topo">
      <h1>Publicações internas</h1>
      <nav class="acoes-topo">
        <a routerLink="/perfil">Meu perfil</a>
        <button type="button" class="btn-sair" (click)="sair()">Sair</button>
      </nav>
    </header>

    <form class="cartao form-criar" [formGroup]="formularioNovo" (ngSubmit)="criar()">
      <h2>Nova publicação</h2>
      <input type="text" formControlName="title" placeholder="Título" />
      <textarea formControlName="body" placeholder="Conteúdo" rows="2"></textarea>
      <button type="submit">Publicar</button>
    </form>

    <div class="cartao busca">
      <input type="text" [formControl]="buscaControl" placeholder="Buscar publicações..." />
    </div>

    @if (postsService.carregando()) {
      <p class="estado">Carregando publicações...</p>
    }

    @if (postsService.erro()) {
      <div class="estado erro">
        <p>{{ postsService.erro() }}</p>
      </div>
    }
  </div>
</div>

```

```

        <button type="button" (click)="postsService.carregar()">Tentar novamente</button>
    </div>
}

@if (!postsService.carregando() && !postsService.erro() && postsService.postsExibidos().length
=== 0) {
    <p class="estado">Nenhuma publicação encontrada.</p>
}

<ul class="lista-posts">
    @for (post of postsService.postsExibidos(); track post.id) {
        <li class="cartao post">
            @if (editandoId() === post.id) {
                <form class="form-editar" [formGroup]="formularioEdicao"
                (ngSubmit)="salvarEdicao(post.id)">
                    <input type="text" formControlName="title" />
                    <textarea formControlName="body" rows="2"></textarea>
                    <div class="acoes">
                        <button type="submit">Salvar</button>
                        <button type="button" (click)="cancelarEdicao()">Cancelar</button>
                    </div>
                </form>
            } @else {
                <h3>{{ post.title }}</h3>
                <p>{{ post.body }}</p>
                <div class="acoes">
                    <button type="button" (click)="iniciarEdicao(post)">Editar</button>
                    <button type="button" class="btn-excluir"
                    (click)="excluir(post.id)">Excluir</button>
                </div>
            }
        </li>
    }
</ul>

@if (postsService.total() > 0) {
    <div class="paginacao">
        <button
            type="button"
            [disabled]="!postsService.temPaginaAnterior()"
            (click)="postsService.paginaAnterior()"
        >
            ← Anterior
        </button>
        <span>Página {{ postsService.paginaAtual() }} de {{ postsService.totalPaginas() }}</span>
        <button
            type="button"
            [disabled]="!postsService.temProximaPagina()"
            (click)="postsService.proximaPagina()"
        >

```

```

        Próxima →
      </button>
    </div>
  }
</div>

@if (toast(); as t) {
  <div class="toast" [class.erro]="t.tipo === 'erro'">{{ t.texto }}</div>
}
</div>

```

4.18 src/app/features/feed/feed.css

Estilos do Feed: cartões brancos para o cabeçalho, formulário e busca; lista de publicações com botões de ação; controles de paginação centralizados; e um toast fixo no rodapé, verde para sucesso e vermelho para erro.

Arquivo: src/app/features/feed/feed.css

```

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

:host {
  display: block;
  background: #f5f7fb;
  min-height: 100vh;
}

.container {
  display: flex;
  justify-content: center;
  padding: 40px 20px;
}

.pagina {
  width: 100%;
  max-width: 640px;
  display: flex;
  flex-direction: column;
  gap: 16px;
}

.cartao {
  background: white;
  padding: 20px;
}

```

```
border-radius: 12px;
box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
}
```

```
.topo {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

```
h1 {
  font-size: 20px;
  color: #222;
}
```

```
h2 {
  font-size: 15px;
  color: #222;
  margin-bottom: 4px;
}
```

```
.acoes-topo {
  display: flex;
  align-items: center;
  gap: 14px;
}
```

```
.acoes-topo a {
  color: #2563eb;
  font-size: 14px;
  font-weight: bold;
  text-decoration: none;
}
```

```
.btn-sair {
  border: none;
  background: #fee2e2;
  color: #b91c1c;
  padding: 8px 16px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
  font-size: 13px;
}
```

```
.btn-sair:hover {
  background: #fecaca;
}
```



```
.form-criar,  
.form-editar {  
  display: flex;  
  flex-direction: column;  
  gap: 10px;  
}  
  
input,  
textarea {  
  padding: 10px 12px;  
  border: 1px solid #ccc;  
  border-radius: 8px;  
  font-size: 14px;  
  font-family: inherit;  
  resize: vertical;  
}  
  
button {  
  border: none;  
  background: #2563eb;  
  color: white;  
  padding: 10px 16px;  
  border-radius: 8px;  
  cursor: pointer;  
  font-weight: bold;  
  font-size: 13px;  
  align-self: flex-start;  
}  
  
button:hover:not(:disabled) {  
  background: #1d4ed8;  
}  
  
button:disabled {  
  background: #93c5fd;  
  cursor: not-allowed;  
}  
  
.estado {  
  text-align: center;  
  color: #666;  
  font-size: 14px;  
  padding: 10px;  
}  
  
.estado.erro {  
  display: flex;  
  flex-direction: column;  
  align-items: center;
```

```
gap: 8px;
color: #dc2626;
font-weight: bold;
}

.lista-posts {
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.post h3 {
  font-size: 15px;
  color: #222;
  margin-bottom: 6px;
}

.post p {
  color: #444;
  font-size: 14px;
  margin-bottom: 10px;
}

.acoes {
  display: flex;
  gap: 8px;
}

.btn-excluir {
  background: #fee2e2;
  color: #b91c1c;
}

.btn-excluir:hover {
  background: #fecaca;
}

.paginacao {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 16px;
  font-size: 13px;
  color: #444;
}

.toast {
  position: fixed;
```

```

bottom: 24px;
left: 50%;
transform: translateX(-50%);
background: #16a34a;
color: white;
padding: 12px 20px;
border-radius: 8px;
font-weight: bold;
font-size: 14px;
box-shadow: 0 4px 18px rgba(0, 0, 0, 0.15);
}

.toast.erro {
  background: #dc2626;
}

```

4.19 src/app/app.ts

O componente raiz. O Angular CLI gera esses arquivos com um template de boas-vindas — substitua o conteúdo por uma versão mínima, já que quem decide o que aparece na tela agora são as rotas.

Arquivo: src/app/app.ts

```

import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  imports: [RouterOutlet],
  selector: 'app-root',
  styleUrls: ['./app.css'],
  templateUrl: './app.html',
})
export class App {}

```

4.20 src/app/app.html

Arquivo: src/app/app.html

```
<router-outlet />
```

5. Executando e testando a aplicação

Com todos os arquivos salvos, suba o servidor de desenvolvimento:

```
$ npm start
```

Ou, de forma equivalente:

```
$ ng serve
```

Abra o navegador em <http://localhost:4200> e teste os cenários abaixo.

Roteiro de teste — Login e Perfil

- Ao abrir a página, você é redirecionado automaticamente para `/login` (rota raiz → `redirectTo: 'login'`).
- Deixe um campo em branco e clique em Entrar → aparece "Preencha usuário e senha.", sem nenhuma requisição sendo enviada.
- Preencha com username emilys e password errada → a API responde 400 e aparece "Usuário ou senha inválidos."
- Preencha com username emilys e password emilyspass → login bem-sucedido, você é levado para `/perfil`, e a seção "Chamada autenticada a `/auth/me`" carrega o papel (role) do usuário.
- Clique em "Ver publicações" → navega para `/feed`.

Roteiro de teste — Feed de publicações

- A lista de publicações carrega automaticamente ao abrir `/feed`, já paginada (10 por página).
- Digite algo no campo de busca (ex.: "love") e aguarde — depois de parar de digitar, a lista é filtrada pela API, sem precisar clicar em nenhum botão.
- Apague o texto da busca → a lista volta a mostrar todas as publicações, da primeira página.
- Use os botões "Anterior" e "Próxima" para navegar entre as páginas — eles ficam desabilitados nos extremos.
- Preencha "Nova publicação" e clique em Publicar → a publicação aparece no topo da lista e um toast verde confirma o sucesso.
- Clique em "Editar" nessa mesma publicação recém-criada, altere o texto e clique em Salvar → a lista é atualizada e o toast de sucesso aparece, sem passar pela API (esse post só existe localmente, veja o capítulo 2).
- Clique em "Excluir" nessa publicação e confirme → ela desaparece da lista com o toast de sucesso.
- Agora clique em "Editar" numa publicação que já existia (não foi criada por você) e salve uma alteração → a mesma operação, mas desta vez passando por um PATCH real para a API.
- Tente acessar <http://localhost:4200/feed> direto pela URL, sem estar logado (ou depois de clicar em Sair) → o `AuthGuard` bloqueia e redireciona para `/login`.
- Com a sessão ativa, abra o DevTools, vá em Application > Session Storage e altere manualmente o valor salvo em `dummyjson-usuario`, trocando o `accessToken` por um

texto qualquer. Recarregue /feed → a chamada à API falha com 401, e o interceptor desloga e redireciona para /login automaticamente.

6. Testes automatizados

O Angular 22 usa o Vitest como executor de testes por padrão.

6.1 src/app/app.spec.ts

Como o App depende do Router e do HttpClient (por causa das rotas lazy e do interceptor), o teste precisa fornecer `provideRouter(routes)`, `provideHttpClient()` e `provideHttpClientTesting()` para que o TestBed consiga montar o componente sem acessar a internet de verdade.

Arquivo: `src/app/app.spec.ts`

```
import { TestBed } from '@angular/core/testing';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { provideHttpClientTesting } from '@angular/common/http/testing';

import { App } from './app';
import { routes } from './app.routes';

describe('App', () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App],
      providers: [provideRouter(routes), provideHttpClient(), provideHttpClientTesting()],
    }).compileComponents();
  });

  it('should create the app', () => {
    const fixture = TestBed.createComponent(App);
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();
  });
});
```

6.2 src/app/core/services/auth.spec.ts

Verifica se o AuthService é criado corretamente pela injeção de dependência, e se ele começa sem sessão ativa quando não há nada salvo em sessionStorage.

Arquivo: `src/app/core/services/auth.spec.ts`

```
import { TestBed } from '@angular/core/testing';
```

```
import { provideHttpClient } from '@angular/common/http';
import { AuthService } from '../auth';

describe('AuthService', () => {
  let service: AuthService;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideHttpClient()],
    });
    service = TestBed.inject(AuthService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });

  it('should start unauthenticated when there is no saved session', () => {
    expect(service.estaAutenticado()).toBe(false);
  });
});
```

6.3 src/app/core/services/posts.spec.ts

Verifica se o PostsService é criado corretamente e se ele começa com a lista vazia e sem paginação disponível.

Arquivo: src/app/core/services/posts.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { PostsService } from '../posts';

describe('PostsService', () => {
  let service: PostsService;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideHttpClient()],
    });
    service = TestBed.inject(PostsService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });

  it('should start with an empty list and no pagination', () => {
    expect(service.posts()).toEqual([]);
  });
});
```

```
expect(service.temPaginaAnterior()).toBe(false);  
expect(service.temProximaPagina()).toBe(false);  
});  
});
```

6.4 Rodando os testes

```
$ ng test --watch=false
```

O resultado esperado é a mensagem de todos os testes passando ("Tests ... passed").

7. Build de produção

Quando o projeto estiver pronto para publicar, gere a versão otimizada com:

```
$ ng build
```

Os arquivos finais são gerados na pasta dist/DummyJSON, prontos para hospedar em qualquer servidor de arquivos estáticos. Como login, perfil e feed são carregados de forma lazy, o build gera um chunk separado para cada uma dessas telas — reduzindo o tamanho do pacote inicial.

Atenção: como esta aplicação guarda o token em sessionStorage e depende de um Route Guard no front-end, lembre-se de que a proteção real dos dados continua sendo responsabilidade da API — o Guard só evita que a interface mostre uma tela que o usuário não deveria ver.